

RAPTOR Reading Companion

An original guide to the runnable ideas in this learning repository. Read it beside the research paper, then trace the fixtures and code here.

START WITH ONE QUESTION

What can I reach with one more boarding? That question gives a RAPTOR round its meaning. Round zero admits walking. Each later round allows one additional vehicle. Earlier arrival and fewer transfers can disagree, so the useful tradeoff stays visible.

What this document is

This is explanatory material for this repository. Its runnable network uses invented stops, times, and hash-shaped identifiers. It is not an official passenger itinerary, a real timetable feed, or a deployed journey service.

The repository is intentionally self-contained: the explanation points to its own chapters, source modules, fixtures, examples, notebooks, and tests. It distinguishes what the lab implements from broader concepts a production system would need.

Research source: Daniel Delling, Thomas Pajor, and Renato F. Werneck, [Round-Based Public Transit Routing](#), ALLENEX 2012. The original paper supplies the algorithmic foundation; this companion supplies a small, inspectable learning path.

01 / Protect the invariant

A label at stop s in round k is the earliest known arrival using at most k boardings. Copy the prior round before writing improvements. A boarding reads only the prior-round arrival plus the admitted slack; it never reads a label written by a different route in the same round.

Read `docs/01_rounds_labels_and_pareto.md`, then trace `src/round_state.py`, `src/route_scan.py`, and `src/raptor.py`. The path witness travels with an improvement so the result can explain how it was reached.

02 / Follow time and direction

Departure profiles, arrive-by, and last connection are different queries. The lab makes that difference explicit instead of treating a point search as a universal answer.

Departure profiles

For a first-board departure d , access walk w , and boarding slack b , the latest catchable origin readiness is $d - w - b$. `src/profile.py` processes those event thresholds latest to earliest and reuses round labels. Adjacent integer-second intervals merge only when their arrival and boarding signatures agree.

A minute-sampled loop is not the implemented range engine. An exhaustive second-by-second loop appears only in the independent oracle. Walking-only profiles need nonconstant affine arrival pieces;

the restricted lab rejects that unsupported domain explicitly.

Reverse search

Start with the destination deadline. **src/reverse.py** uses incoming indexes of existing forward walking edges, subtracts durations, and scans trips backward. It reasons backward about a legal forward journey; it does not invent a reverse physical walkway.

Last connection

The final feasible journey follows the supplied service date's actual events. It is not defined by 23:59, midnight, or a convenient cutoff. The overnight fixture keeps 24:06:00 as service seconds associated with its original date.

Read docs/03_service_days_and_timetables.md and docs/06_departure_profiles_and_reverse_search.md. Run example_journey_profiles.py to connect these modes to the invented fixture.

03 / Connect the lab

An algorithm result, an admitted input, and a deployed observation are different kinds of evidence. This repository focuses on the first two as a learning model.

The local end-to-end path

docs/04_easysubway_end_to_end.md is a local end-to-end lab chapter: it explains how an input snapshot is validated, compiled into indexes, queried by the router, and projected into a journey result. The chapter is a teaching model, not a pointer to another codebase or a claim about a live system.

The concrete implementation is split by responsibility: **src/timetable.py** and **src/route_index.py** shape and index the fixture; **src/raptor.py** drives rounds; **src/footpaths.py** closes walking paths; **src/journey.py** builds a witness; and **src/accessibility.py**, **src/realtime.py**, and **src/oracle.py** expose the lab's stated constraints.

Prove the restricted model honestly

The independent oracle enumerates boardable trips and walking edges on a state graph. Point tests compare every stop and boarding budget; profile tests cover each integer second of small windows; reverse tests exhaust forward readiness. These are local correctness checks, not performance evidence for a production deployment.

Try a counterexample before an optimization: reverse two connecting routes, add one second to a ready time, or supply a walking chain in reverse edge order. The answer should follow the semantics, not iteration luck.

The study boundary

Implemented here: one admitted service date, static walking, arrival and boarding routing, restricted range and last-connection domains, an illustrative realtime overlay, and explicit failures for unsupported cases. Explained as extensions: full feed ingestion, cryptographic artifact admission, unrestricted multicriteria routing, public endpoints, user interface policy, and deployment activation.

04 / Read, run, and extend

Use the repository as a sequence of claims you can inspect and challenge.

A practical study loop

1. Read **docs/00_why_transit_needs_raptor.md** for the query decision table. 2. Trace the invented network in **README.md** and run **example_routing.py**. 3. Follow the corresponding modules under **src/**. 4. Inspect the named test for the invariant it checks. 5. Use the notebooks to compare labels, profiles, and frontiers.

The frontier limit

A single earliest-arrival label is sound only under this lab's fixed walking duration, fixed slack, waiting, and arrival-time-plus-boardings objective. If the product promises least walking, connection safety, or path-dependent accessibility, it needs a richer frontier. **src/pareto.py** demonstrates dominance and fails on capacity loss; it is not a complete multicriteria route scan.

Research and timetable references

Read the [official RAPTOR publication](#) for the route-scan algorithm and the [GTFS Schedule reference](#) for service times beyond 24 hours. The repository models those ideas with deliberately small fixtures; it does not ingest GTFS feeds.

Keep the claim proportional to the evidence

When extending the lab, first state the new input contract, state representation, objective, failure behavior, and an independent check. Do not let a convenient fixture or a passing example silently become a general routing guarantee.

Next reading: **docs/05_transfers_accessibility_and_identity.md**, **docs/07_multicriteria_frontiers_and_extensions.md**, and **docs/08_correctness_performance_and_study_plan.md**.